

# Executive Summary

---

**Core Recommendation:** Use **Explicit Graph RAG (Neo4j + Weaviate)** for production. Use Implicit Graph RAG (Weaviate-only) **ONLY** for the MVP/PoC phase. The primary reason: industrial safety systems require structurally guaranteed causal chain traversal — something embedding similarity alone cannot provide.

## The Core Problem

CHP training AI must answer multi-hop queries like: "If the Belt Conveyor trips, what downstream systems are at risk and what is the correct operator response?" This demands causal reasoning across the chain: Belt Conveyor → Coal Bunker → Boiler → Unit Trip. Pure vector search cannot reliably traverse this — cross-domain embeddings have low similarity.

## Why Explicit Graph RAG Wins

Neo4j enforces structural integrity. Causal chains, failure propagation paths, and system dependencies are stored as first-class graph edges — not inferred probabilistically. In safety-critical systems, a wrong answer is not just unhelpful — it can lead to unsafe actions.

## Bi-Modal Alignment Is Non-Negotiable

Text chunks AND SVG/image diagrams must be linked to the same graph node when similarity  $\geq 0.8$ . Below this threshold, NEW nodes and relationships must be created. This 0.8 rule prevents hallucination from incorrect node associations and is the key design constraint.

# 1. Data Model — Excel KG + HTML Modules

## 1.1 Excel Knowledge Graph — Node & Relationship Structure

The CHP\_WRCF.xlsx file defines the graph backbone across 5 sheets:

| Sheet         | Node Type                     | Key Attributes                                                                      | Graph Role                                |
|---------------|-------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------|
| FG & PWO      | FunctionalGroup<br>WorkObject | Strategic Importance, Revenue Impact,<br>Downtime Sensitivity, Dependency Links     | Top-level domain hierarchy                |
| SWO           | SecondaryWorkObj<br>ect       | Operational Complexity, Asset Criticality,<br>Failure Frequency                     | Sub-system nodes with failure<br>metadata |
| Skills        | Skill, Capability             | Capability Code, Proficiency Level,<br>Cognitive Type, Skill Criticality, AI Impact | Competency nodes linked to<br>SWOs        |
| Tasks         | Task                          | Frequency, Complexity, Duration, Required<br>Proficiency                            | Operational task nodes                    |
| Control Point | ControlPoint                  | Risk Level, Failure Impact, KPI Threshold,<br>Escalation Required, Evidence Type    | Safety/quality gate nodes                 |

## 1.2 Graph Node Hierarchy

|                                                                                 |
|---------------------------------------------------------------------------------|
| Industry (Thermal Power Plant)                                                  |
| ■ ■ ■ FunctionalGroup (Coal Handling System, Boiler System, Turbine System ...) |
| ■ ■ ■ PrimaryWorkObject (Coal Receipt & Unloading, Coal Conveying System ...)   |
| ■ ■ ■ SecondaryWorkObject (Belt Conveyor, Track Hopper, Wagon Tippler ...)      |
| ■ ■ ■ Skill (CAP-01 to CAP-09)                                                  |
| ■ ■ ■ ■ Task (Daily/Weekly operational tasks with complexity/duration)          |
| ■ ■ ■ ControlPoint (Risk gates: KPI threshold, escalation, evidence type)       |

## 1.3 HTML Training Module — Semantic Chunk Types

Four chunk types are extracted from HTML — never splitting mid-concept:

| Chunk Type     | HTML Source                             | Embedding Model           | Weaviate Class |
|----------------|-----------------------------------------|---------------------------|----------------|
| ConceptChunk   | , h2 + content blocks                   | text-embedding-3-large    | TextChunk      |
| ProcedureChunk | Execution Block divs, numbered steps    | text-embedding-3-large    | TextChunk      |
| RiskChunk      | Warning boxes, Control Point tables     | text-embedding-3-large    | TextChunk      |
| DiagramChunk   | SVG elements, inline diagram containers | CLIP + GPT-4V description | ImageChunk     |

## 2. Architecture Design

### 2.1 Explicit Graph RAG — Neo4j + Weaviate

#### Architecture Flow (Mermaid):

```
INGESTION:
Excel KG [Parse nodes/edges] -> Neo4j Graph DB
HTML Modules [Semantic chunker] -> Text Chunks [embed] -> Weaviate TextChunk
HTML Modules [SVG extractor] -> Image Chunks [CLIP+GPT4V] -> Weaviate ImageChunk
TextChunk / ImageChunk [sim >= 0.8] -> Linked to Neo4j node
TextChunk / ImageChunk [sim < 0.80] -> NEW Node created in Neo4j
```

```
QUERY:
User Query -> NLP Entity Extractor -> Neo4j Cypher (graph traversal)
Neo4j [Relevant node_ids] -> Weaviate filter retrieval
Weaviate [Top-k text + image chunks] -> LLM Context Assembly
LLM -> Grounded Answer + Diagram references
```

**Neo4j:** Stores full graph: FG → PWO → SWO → Skill → Task → ControlPoint hierarchy plus CAUSES, DEPENDS\_ON, FLOWS\_TO causal edges.

**Weaviate:** Vector store for all text and image chunks. Each chunk carries node\_id FK back to Neo4j. Supports hybrid search (vector + keyword + metadata filter).

**Entity Extractor:** Detects CHP entities from user query (e.g., "Belt Conveyor", "coal bunker starvation") and maps to Neo4j node IDs.

**Cypher Traversal:** Executes multi-hop: MATCH (n)-[:CAUSES\*1..3]->(m) to find failure propagation paths, upstream deps, and safety control points.

**Bi-modal Retrieval:** Uses node\_id list from Neo4j to filter Weaviate, then retrieves top-k text + image chunks ranked by cosine similarity.

### 2.2 Implicit Graph RAG — Weaviate-Only + FastAPI

```
INGESTION:
Excel KG [Convert to RelationObject JSON] -> Weaviate RelationChunk
HTML Modules [Semantic chunker + embed] -> Weaviate TextChunk (with metadata)
HTML Modules [SVG + CLIP embed] -> Weaviate ImageChunk

QUERY:
User Query [embed] -> Weaviate vector search -> Top-k chunks
Follow RelationChunk source_id/target_id links [Python loop, max 2-3 hops]
Re-rank all chunks -> LLM Context -> Answer
```

In this approach, Excel relationships are stored as structured Weaviate objects with source\_id, target\_id, relation\_type, and description fields. Multi-hop traversal is simulated by iterative Python loops — not by a native graph database engine.

### 3. Bi-Modal Retrieval Strategy

#### 3.1 The 0.8 Similarity Threshold Rule

**Core Design Rule:** A text or image chunk is linked to an existing graph node ONLY if cosine similarity  $\geq$  0.8. Below 0.8, a NEW node and relationship are created. This prevents hallucination from incorrect node associations.

| Similarity  | Action                                        | CHP Example                                                                  |
|-------------|-----------------------------------------------|------------------------------------------------------------------------------|
| $\geq 0.95$ | Link to existing node (high confidence)       | "Belt Conveyor startup procedure" $\rightarrow$ Belt Conveyor Operation node |
| 0.80 – 0.94 | Link to existing node (acceptable)            | Diagram of "conveyor cross-section" $\rightarrow$ Belt Conveyor node         |
| 0.60 – 0.79 | Create child node, link with HAS_CONTEXT      | "Idler spacing calculation" $\rightarrow$ new BeltConveyor_IdlerDetail node  |
| $< 0.60$    | Create standalone node, flag for human review | "Dust extraction chemistry" $\rightarrow$ new DustSuppression_Chemistry node |

#### 3.2 Image / Diagram Ingestion Pipeline

**Step 1 — Extract SVG/Image:** Parse HTML for elements. Extract embedded text labels, element IDs, flow arrows, and structural metadata.

**Step 2 — Generate Description:** Use GPT-4V / Claude Vision to generate structured NL description: component labels, flow directions, highlighted fault states, and system name.

**Step 3 — Text-Compatible Embedding:** Embed the generated description using text-embedding-3-large to create a text-compatible image embedding for cross-modal similarity.

**Step 4 — CLIP Embedding:** Also generate CLIP visual embedding for image-to-image retrieval. Store both embeddings in Weaviate ImageChunk.

**Step 5 — Node Linking:** Compare description embedding vs. all node vectors. Apply 0.8 threshold: link or create new node.

**Step 6 — Store Metadata:** source\_module, heading\_context, section\_id, node\_ids[], diagram\_type (flow/schematic/risk\_matrix), svg\_raw.

#### 3.3 Query-Time Bi-Modal Fusion

1. Embed user query  $\rightarrow$  query\_vector
2. Neo4j: entity match + Cypher traversal  $\rightarrow$  relevant node\_ids[]
3. Weaviate TextChunk: filter node\_ids, rank cosine similarity  $\rightarrow$  top-5 text chunks
4. Weaviate ImageChunk: filter node\_ids, rank cosine similarity  $\rightarrow$  top-3 image chunks
5. Context: [System Prompt] + [Graph Path Summary] + [Text Chunks] + [Diagram Descriptions]
6. Visual grounding: include SVG raw or diagram description inline in LLM prompt
7. LLM answer cites: (Source: Belt\_Conveyor\_L1, Section 3, ControlPoint CP-05)

## 4. Query Execution — Step by Step

---

**Example Query: "Belt Conveyor trips — what fails downstream and what should the operator do?"**

### 4.1 Explicit Graph RAG Execution

#### Step 1 — Entity Extraction

```
NLP extracts: entity="Belt Conveyor", intent="failure propagation", action_required="operator response"
```

#### Step 2 — Neo4j Node Lookup

```
MATCH (n:SWO {name: "Belt Conveyor Operation"}) RETURN n, n.node_id
```

#### Step 3 — Causal Graph Traversal

```
MATCH (n)-[:FLOWS_TO|CAUSES*1..3]->(d) RETURN d → Result: BeltConveyor → CoalBunker → BoilerFeed → BoilerShutdown
```

#### Step 4 — Control Point Retrieval

```
MATCH (n)-[:HAS_CONTROL_POINT]->(cp) WHERE cp.risk_level="High" RETURN cp → Conveyor Trip Control Point, Isolation Procedure, Escalation Required=YES
```

#### Step 5 — Weaviate Filter Search

```
Filter TextChunk+ImageChunk by node_ids from Step 2-4 → top-5 text chunks + top-2 diagram chunks (conveyor trip schematic)
```

#### Step 6 — Context Assembly

```
Graph path summary + control points + text chunks + diagram descriptions → LLM prompt
```

#### Step 7 — LLM Generation

```
Output: causal explanation + operator checklist + diagram reference + KPI thresholds from ControlPoint sheet
```

### 4.2 Implicit Graph RAG — Same Query (Contrast)

#### Step 1 — Embed Query

```
"Belt Conveyor trip downstream failure operator response" → query_vector
```

#### Step 2 — Vector Search

```
Top-10 TextChunks by cosine similarity (NO graph filter — all chunks eligible)
```

#### Step 3 — Relation Lookup

```
Find RelationChunks where source matches "Belt Conveyor" Follow target_id links iteratively in Python (max 2 hops)
```

#### Step 4 — Risk

```
If "Coal Bunker starvation" embedding is not close to "Belt Conveyor trip", the causal chain is BROKEN. Boiler and Turbine consequences may NOT be retrieved.
```

## Step 5 — LLM Generation

May generate plausible but incomplete answer – missing downstream boiler/unit trip risk

**Critical Failure:** In Implicit Graph RAG, if "Coal Bunker starvation" is not semantically near "Belt Conveyor trip" in embedding space, the retrieval MISSES the causal chain. In a real CHP plant, the operator may not know the boiler is at risk. This is unacceptable in a safety-critical system.

## 5. Full Architecture Comparison

| Dimension                   | Explicit Graph RAG (Neo4j + Weaviate)                                   | Implicit Graph RAG (Weaviate-Only)                                                |
|-----------------------------|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Architecture                | Neo4j + Weaviate + FastAPI + LLM                                        | Weaviate (relations as objects) + FastAPI + LLM                                   |
| Graph Storage               | Native graph: nodes, typed edges, Cypher query engine                   | Simulated: relation JSON objects with source_id/target_id fields                  |
| Multi-hop Reasoning         | Native: MATCH (a)-[:CAUSES*1..3]->(b)<br>Guaranteed traversal           | Python loops, max 2-3 hops Fragile at scale                                       |
| Causal Chain (CHP)          | Belt→Bunker→Boiler always resolved Graph edge is source of truth        | Chain broken if embeddings not close No structural guarantee                      |
| Bi-modal Linking            | Text + image linked via 0.8 threshold Structural alignment enforced     | Metadata fields only No structural integrity check                                |
| Graph Expansion             | New Neo4j nodes/edges from HTML Fully traceable lineage                 | New RelationChunk objects only Not queryable as true graph                        |
| Hallucination Risk          | LOW — graph constraints limit retrieval to structurally related nodes   | MEDIUM-HIGH — similarity-based only Plausible but causally wrong content possible |
| Query Latency               | 1.0 – 2.1 seconds (Cypher + Weaviate)                                   | 0.8 – 1.8 seconds (single vector search)                                          |
| Infrastructure Cost         | \$470 – \$1,050 / month (cloud)                                         | \$270 – \$650 / month (cloud)                                                     |
| Dev Complexity              | HIGH: graph schema, Cypher, bidirectional sync, ingestion pipeline      | MEDIUM: JSON schema, Python loops, no graph DB expertise needed                   |
| Safety-Critical Suitability | HIGH — structural guarantees, audit trails, explainable paths           | MEDIUM — OK for training content Q&A, NOT for failure response guidance           |
| MVP Speed                   | 4–6 weeks to working pipeline                                           | 2–3 weeks to working PoC                                                          |
| Ordering / Sequences        | Step edges guarantee correct sequence (NEXT_STEP graph edges)           | Embedding retrieval CANNOT guarantee step ordering                                |
| Audit / Compliance          | Full traceability: answer → chunk → node → graph edge → source document | Weak: chunk → source doc only No graph path audit trail                           |
| RECOMMENDATION              | USE FOR PRODUCTION                                                      | USE FOR MVP/PoC ONLY                                                              |

## 6. Multi-Hop & Causal Reasoning — CHP Scenarios

### 6.1 Failure Propagation — Belt Conveyor Trip

This is the definitive test case for any CHP RAG system. The full causal chain:

| System Node                  | Failure Consequence         | KPI Trigger          | Escalation         |
|------------------------------|-----------------------------|----------------------|--------------------|
| Belt Conveyor BC-01 trips    | Coal feed to bunker stops   | Conveyor current = 0 | Alert operator     |
| Coal Bunker starvation       | Level drops below threshold | < 30% level          | Supervisor alert   |
| Boiler coal feed interrupted | Steam generation drops      | < 85% MCR            | Shift Engineer     |
| Steam pressure drops         | Turbine load reduction      | < rated pressure     | Plant Manager      |
| Unit Trip                    | Grid load shedding          | Zero generation      | Emergency protocol |

***Explicit Graph RAG** executes: `MATCH (bc:SWO {name:"Belt Conveyor"})-[:CAUSES*1..5]->(m) RETURN m, m.kpi_threshold`. Every hop is guaranteed. KPI values come from ControlPoint nodes. Escalation chain comes from control point escalation flags.*

***Implicit Graph RAG:** "Steam pressure drop" and "belt conveyor trip" have near-zero embedding similarity (mechanical vs. thermal domain). The Python hop loop will NOT traverse this chain reliably. The LLM may generate a partial answer or invent plausible but incorrect consequences.*

### 6.2 Where Weaviate-Only Definitively Breaks Down

**Cross-domain causality:** Belt Conveyor (mechanical) → Boiler (thermal) → Turbine (electrical). Low cross-domain embedding similarity. Graph edges are the ONLY reliable connector.

**Negative inference:** "What should NOT happen if the feeder starts?" requires knowing all control points linked to a node and their inverse conditions. Vector search cannot do this.

**Ordered traversal:** "Correct startup sequence for Coal Receipt system" requires ordered edges (STEP\_1 → STEP\_2 → STEP\_3). Embeddings retrieve all steps but cannot order them.

**Completeness guarantee:** Safety systems require ALL control points for a given SWO — not just the top-k most similar. Vector search may miss a critical safety gate with slightly different wording.

## 7. Graph Expansion from HTML Content

---

When HTML training modules contain knowledge not in the Excel KG, new nodes and edges are created:

### Step 1 — Entity Extraction

NLP extracts CHP-specific terms from chunk: equipment names, failure modes, KPIs not yet in graph.

### Step 2 — Embed & Compare

Compute cosine similarity of chunk embedding against all existing Neo4j node vectors.

### Step 3 — $\geq 0.8$

LINK: Create HAS\_TEXT\_CHUNK or HAS\_IMAGE\_CHUNK edge from existing node. Update node metadata with source\_module, proficiency\_level.

### Step 4 — 0.6–0.79

CREATE CHILD NODE: e.g., BeltConveyor\_IdlerDetail linked to parent with HAS\_DETAIL edge. Chunk linked to new node. Flagged for review.

### Step 5 — $< 0.60$

CREATE STANDALONE NODE: Auto-classify type (Equipment/Procedure/Risk/Control). Link to best-matching FG/PWO parent. Requires human approval.

### Step 6 — Image Expansion

Same 0.8 rule for diagram description embeddings. New diagram showing unrecognized system → EquipmentVariant node created.

### Step 7 — Audit Trail

All auto-created nodes carry: source="html\_expansion", confidence\_score, created\_date, review\_status. Non-destructive — Excel KG nodes never modified.

**Design Principle:** Graph expansion is ADDITIVE and NON-DESTRUCTIVE. Excel KG nodes are the authoritative source — HTML expansion only ADDS new nodes and edges. All auto-created nodes require human review before serving safety-critical queries.

## 8. Performance, Cost & Infrastructure

### 8.1 Query Latency Breakdown

| Operation                      | Explicit Graph RAG       | Implicit Graph RAG              |
|--------------------------------|--------------------------|---------------------------------|
| Entity extraction              | 50–100ms                 | 50–100ms                        |
| Graph traversal (Neo4j Cypher) | 100–300ms                | N/A                             |
| Vector search (Weaviate)       | 50–150ms (node-filtered) | 80–200ms (full collection scan) |
| Context assembly               | 20–50ms                  | 50–150ms (Python hop loops)     |
| LLM generation                 | 800–1500ms               | 800–1500ms                      |
| TOTAL                          | ~1.0–2.1 seconds         | ~1.0–1.9 seconds                |

### 8.2 Cloud Infrastructure Cost Estimate

| Component       | Explicit Graph RAG                              | Implicit Graph RAG                              |
|-----------------|-------------------------------------------------|-------------------------------------------------|
| Graph Database  | Neo4j AuraDB Professional<br>~\$200–400/month   | Not required — \$0                              |
| Vector Database | Weaviate Cloud Standard<br>~\$100–200/month     | Weaviate Cloud Standard<br>~\$100–200/month     |
| API / Compute   | FastAPI on Cloud Run ~\$50–100/month            | FastAPI on Cloud Run ~\$50–100/month            |
| Embedding API   | OpenAI text-embedding-3-large<br>~\$20–50/month | OpenAI text-embedding-3-large<br>~\$20–50/month |
| LLM API         | Claude Sonnet / GPT-4o<br>~\$100–300/month      | Claude Sonnet / GPT-4o<br>~\$100–300/month      |
| TOTAL           | \$470–\$1,050/month                             | \$270–\$650/month                               |

## 9. Failure Cases & Limitations

---

### 9.1 Explicit Graph RAG — Known Failure Points

**Entity Matching Failure:** "BC-01 trip" may not match Neo4j node "Belt Conveyor Operation" without fuzzy matching. Mitigation: alias table in Neo4j + fallback vector search.

**Schema Rigidity:** New equipment types not in Excel KG require schema migration. Cannot auto-infer new node types without human-in-the-loop.

**Sync Lag:** HTML updates not reflected in Neo4j until re-indexed. Requires automated change detection + ingestion pipeline.

**Cypher Complexity:** Complex multi-hop queries can time out on large graphs. Requires query optimization, index tuning, and hop-depth limits.

**Cold Start Effort:** Full ingestion pipeline (Excel + HTML + images) takes 2–3 weeks to build. Higher initial investment.

### 9.2 Implicit Graph RAG — Known Failure Points

**Causal Chain Breaks:** Cross-domain causal chains (mechanical→thermal→electrical) have low embedding similarity. Python hop loops will not traverse reliably.

**Completeness Gap:** Top-k vector retrieval may omit critical control points or safety procedures that are worded differently from the query.

**Hallucination Risk:** LLM may infer plausible but causally incorrect relationships from semantically similar but structurally unrelated content.

**Relation Object Sprawl:** Weaviate RelationChunk collection grows unbounded, degrading retrieval quality as dataset scales.

**Step Ordering Impossible:** Startup sequences and escalation ladders require ordered steps. Embedding retrieval cannot guarantee order without metadata hacks.

**Audit Trail Weak:** For safety compliance (ISO, OSHA, plant SOPs), every answer must trace to a specific procedure node. Implicit graph cannot provide this path.

## 10. Final Recommendation & Roadmap

---

### 10.1 MVP Phase — Weeks 1 to 6

**Use Implicit Graph RAG (Weaviate-Only).** Goal: validate content quality, chunking strategy, bi-modal retrieval, and UI/UX. Focus on L1/L2 training content Q&A; where semantic retrieval is sufficient. Accept that multi-hop causal reasoning will be limited — document known gaps explicitly.

Week 1–2: Semantic chunker for HTML (topic-block level, no mid-concept splits). Test on all 4 HTML files.  
Week 2–3: Excel KG → RelationChunk objects in Weaviate. Implement 0.8 threshold node-linking logic.  
Week 3–4: SVG/image ingestion: GPT-4V descriptions + text-compatible embeddings + CLIP embeddings.  
Week 4–5: FastAPI query service. Python-based 2-hop traversal. Basic bi-modal context assembly.  
Week 5–6: UI integration. Evaluate on 50 test queries. Document causal chain failure cases explicitly.

### 10.2 Production Phase — Weeks 7 to 16

**Migrate to Explicit Graph RAG (Neo4j + Weaviate).** This is non-negotiable for a safety-critical industrial system. Failure propagation guidance, escalation procedures, and control point retrieval must be structurally guaranteed — not probabilistically estimated.

Week 7–9: Neo4j schema design. Migrate Excel KG. Implement Cypher query templates for all 5 query types.  
Week 9–11: Update ingestion: dual write — Neo4j nodes + Weaviate chunks with node\_id FK. Auto-sync pipeline.  
Week 11–13: CHP domain entity extractor with ontology + equipment alias table for code-based lookups.  
Week 13–14: Full bi-modal pipeline with 0.8 threshold image linking and new node auto-creation.  
Week 14–15: Graph expansion pipeline: auto node creation from HTML, human review queue, audit flags.  
Week 15–16: Load testing, latency optimization, compliance audit, production deployment.

### 10.3 Non-Negotiable Design Rules (Both Phases)

NEVER split a semantic chunk mid-topic. Each chunk = one complete concept block from HTML.  
ALWAYS apply 0.8 similarity threshold for node linking. Do not force incorrect associations.  
ALWAYS link image chunks to the same node as surrounding text when similarity  $\geq 0.8$ .  
ALWAYS store source\_module, section\_id, proficiency\_level metadata on every chunk.  
NEVER return failure response/operator procedure answers without a ControlPoint node citation.  
ALWAYS include diagram description (not just text) when answering system flow questions.  
Auto-created nodes MUST carry confidence\_score and source="html\_expansion" flag for auditability.

---

*This document is grounded in the actual CHP\_WRCF.xlsx knowledge graph (FG & PWO, SWO, Skills, Tasks, Control Point sheets) and HTML training module structure (Track Hopper L2, Belt Conveyor L1, iCard Microlearning). All architecture decisions reflect real data characteristics and industrial CHP domain requirements.*